

Simulado Avançado — 20 Questões (nível acima das Listas 1 e 2)

Cobre: trace manual, corte, findall/forall, recursão em listas, jarros d'água, árvores binárias, FNC, notação de Kowalski e relações de família. Tente resolver tudo **antes** de olhar o gabarito no final.

Questão 1. Qual é a saída (e o valor de L) da consulta abaixo?

```
d(1). d(2). d(3).  
?- findall(X, (d(X), X > 1), L).
```

Questão 2. Considere:

```
teste(X) :- X > 5, !, write(grande).  
teste(_) :- write(pequeno).
```

Qual a saída de `?- teste(3).`? E de `?- teste(3), fail.`?

Questão 3. Escreva o predicado `intercala/3` que intercala os elementos de duas listas de mesmo tamanho, elemento a elemento. Exemplo:

```
?- intercala([1,3,5],[2,4,6],L).  
L = [1,2,3,4,5,6].
```

Questão 4. Escreva o predicado `achatar/2` (flatten) que transforma uma lista de listas aninhadas em uma lista simples, sem usar o predicado embutido `flatten/2`. Exemplo:

```
?- achatar([1,[2,3],[4,[5,6]],7],L).  
L = [1,2,3,4,5,6,7].
```

Questão 5. Há dois jarros, agora de capacidade **5** e **6** litros. Escreva as duas cláusulas de `transforma/3` referentes a **encher** cada jarro e as duas referentes a **esvaziar**, seguindo o padrão `[X,Y] = [jarro5, jarro6]`.

Questão 6. Dado o par de jarros de 5 e 6 litros do exercício anterior, se o estado atual é `[3,4]`, qual é o estado resultante de `transforma('transferir do jarro 1 para o 2', [3,4], S)`? Mostre o raciocínio (quanto cabe no jarro 2 antes de transbordar).

Questão 7. Considere a árvore representada como no material:

```
no(a,b,c). no(b,d,[]). no(d,[],[]).  
no(c,e,f). no(e,[],g). no(f,[],[]). no(g,[],[]).
```

Escreva `folhas/2`, que retorna a lista de nós-folha (nós sem filhos, ou seja, `no(X,[],[])`) usando percurso pré-ordem.

Questão 8. Para a mesma árvore da questão 7, escreva `altura/2`, que calcula a altura da árvore (número de arestas do caminho mais longo até uma folha). Considere altura de uma folha = 0.

Questão 9. Considere a árvore binária de busca representada como `no(Info,Esq,Dir)` (segunda representação do material, com `insere/3`). Escreva `espelha/2`, que inverte a árvore trocando esquerda e direita em todos os nós, recursivamente.

Questão 10. Converta a fórmula proposicional abaixo para a Forma Normal Conjuntiva (FNC), mostrando os passos (eliminação de $\rightarrow$, De Morgan, distributiva):

$(A \rightarrow B) \rightarrow (\neg B \rightarrow \neg A)$

Questão 11. Escreva, em notação de Kowalski (`conclusões  $\leftarrow$  condições`), a cláusula correspondente à fórmula:

$\neg P \quad \neg Q \quad R \quad S$

(assuma que você deve isolar R e S como conclusões).

Questão 12. Dado o conjunto de cláusulas:

$Q \leftarrow R, S$
 $R \leftarrow T$
 $S \leftarrow$
 $T \leftarrow$

Faça a prova por refutação (mostrando cada passo $\leftarrow \dots$) de que Q é consequência do conjunto de cláusulas.

Questão 13. Defina as regras de família a seguir a partir de `pai/2` e `mae/2` (já existentes na base):

`tio_ou_tia(X,Y) :- ...`

X é tio ou tia de Y se X é irmão(ã) de um dos pais de Y (mas não é o próprio pai/mãe de Y). Defina também `irmao_de/2` como predicado auxiliar.

Questão 14. Defina `primos/2` (primos de sangue, “cousins”): X e Y são primos se seus pais são irmãos entre si.

Questão 15. Qual é a saída da consulta abaixo? Explique o papel do corte.

```
conta(_, [], 0).
conta(X, [X|T], N) :- !, conta(X, T, N1), N is N1+1.
conta(X, [_|T], N) :- conta(X, T, N).

?- findall(N, (member(X, [a,b,a,c,a]), conta(X, [a,b,a,c,a], N)), L).
```

Questão 16. Escreva um predicado `triplas_pitagoricas/2` que, dado um limite N, retorna (via `findall`) todas as triplas [A,B,C] com $A \leq B \leq C \leq N$ tais que $A^2 + B^2 = C^2$. Use `between/3`.

Questão 17. Considere:

```
p(1). p(2). p(3).
q(X,Y) :- p(X), p(Y), X \= Y.
```

Quantas soluções tem `?- q(X,Y).` em backtracking completo? Liste-as na ordem em que o Prolog as encontra.

Questão 18. Escreva `maplist`-based: usando `maplist/3`, defina uma consulta que, dada a lista [1,2,3,4], produza L = [1,4,9,16] (quadrados). Você pode definir um predicado auxiliar `quad/2`.

Questão 19. No mundo do aspirador (estado [Pos,Suja1,Suja2]), escreva os `oper/4` (no padrão `oper(Ação,Estado,Estado') :- Condição`) para as três ações: `entrarSala1`, `entrarSala2` e `aspirar` (cobrindo os dois casos: aspirador na sala 1 ou na sala 2).

Questão 20. Considere o predicado dinâmico abaixo:

```
:- dynamic(contador/1).
contador(0).
```

```
incrementa :-
```

```
retract(contador(N)),
N1 is N + 1,
assert(contador(N1)).
```

Se executarmos `incrementa` três vezes seguidas (`?- incrementa, incrementa, incrementa.`), qual será o valor de `contador(X)` ao final? Explique por que `retract/assert` funcionam aqui sem deixar cláusulas duplicadas ou órfãs.

GABARITO

Questão 1

`L = [2, 3]`

`findall` coleta todos os `X` tais que `d(X)` é verdadeiro e `X > 1`. `d(1)` é descartado pelo filtro.

Questão 2

- `?- teste(3).` → `3 > 5` falha, então a **primeira cláusula falha antes do corte** (o corte nunca é executado, pois está depois do teste). Prolog tenta a segunda cláusula: escreve **pequeno**. Saída: **pequeno** seguido de **true**.
 - `?- teste(3), fail.` → mesma saída **pequeno** primeiro; depois o **fail** força backtracking. Como o corte da primeira cláusula nunca foi executado (a cláusula toda falhou antes dele), não há problema — mas também não há mais alternativas para `teste(3)` (só havia duas cláusulas e ambas já foram tentadas), então a consulta finalmente falha: **false**.
-

Questão 3

```
intercala([], [], []).
intercala([H1|T1], [H2|T2], [H1,H2|R]) :- intercala(T1, T2, R).
```

Questão 4

```
achatar([], []) :- !.
achatar([H|T], L) :-
    is_list(H), !,
    achatar(H, L1),
    achatar(T, L2),
```

```

    append(L1, L2, L).
achatar([H|T], [H|L]) :-
    achatar(T, L).

```

Ideia: se a cabeça é uma lista, achata-a recursivamente e concatena com o resultado do resto; senão, mantém o elemento e segue.

Questão 5

```

transforma('encher o jarro 1', [X,Y], [5,Y]) :- X < 5.
transforma('encher o jarro 2', [X,Y], [X,6]) :- Y < 6.

transforma('esvaziar o jarro 1', [X,Y], [0,Y]) :- X > 0.
transforma('esvaziar o jarro 2', [X,Y], [X,0]) :- Y > 0.

```

Questão 6

Estado [3,4], jarro 1 = capacidade 5, jarro 2 = capacidade 6. Transferir 1→2: X=3 (tem água), Y=4 < 6 (não está cheio). $Z \text{ is } X+Y = 7$. Como $Z=7 > 6$ (capacidade do jarro 2), o jarro 2 fica cheio (6) e sobra R is $Z-6 = 1$ no jarro 1.

Resultado: S = [1,6]

Questão 7

```

folhas(X, [X]) :- no(X, [], []), !.
folhas(X, L) :-
    no(X,E,D),
    folhas_sub(E, LE),
    folhas_sub(D, LD),
    append(LE, LD, L).

folhas_sub([], []) :- !.
folhas_sub(X, L) :- folhas(X, L).

```

Para a árvore dada: $\text{folhas}(a, L) \rightarrow L = [d, g, f]$ (d é folha de b; g e f são folhas de c, já que e tem g como filho direito mas nenhum filho esquerdo — atenção: e não é folha porque tem o filho g).

Questão 8

```
altura(X, 0) :- no(X, [], []), !.  
altura(X, H) :-  
    no(X, E, D),  
    altura_sub(E, HE),  
    altura_sub(D, HD),  
    H is max(HE, HD) + 1.
```

```
altura_sub([], -1) :- !.  
altura_sub(X, H) :- altura(X, H).
```

(altura_sub([], -1) garante que um ramo ausente não conte como caminho — somando 1 ele vira 0, correspondente a “sem filho por esse lado”).

Para a árvore dada: $\text{altura}(a, H) \rightarrow H = 2$ ($a \rightarrow c \rightarrow e \rightarrow g$ é o caminho mais longo, com 3 arestas... revise com seu professor a convenção exata de “altura”, pois alguns contam nós e não arestas — aqui usamos arestas, o que dá $H=3$ para $a \rightarrow c \rightarrow e \rightarrow g$. Trace você mesmo para fixar o conceito.)

Questão 9

```
espelha([], []) :- !.  
espelha(no(X, E, D), no(X, DE, ED)) :-  
    espelha(E, ED),  
    espelha(D, DE).
```

Note a troca: o filho esquerdo do resultado (DE) vem do espelhamento do filho **direito** original (D), e vice-versa.

Questão 10

$(A \rightarrow B) \rightarrow (\neg B \rightarrow \neg A)$

Passo 1 — eliminar as duas implicações ($P \rightarrow Q \equiv \neg P \vee Q$):

$$\begin{aligned} &= \neg(A \rightarrow B) \vee (\neg B \rightarrow \neg A) \\ &= \neg(\neg A \vee B) \vee (\neg(\neg B) \vee \neg A) \\ &= \neg(\neg A \vee B) \vee (B \vee \neg A) \end{aligned}$$

Passo 2 — De Morgan em $\neg(\neg A \vee B)$:

$$\neg(\neg A \vee B) = (A \wedge \neg B)$$

Logo:

$$= (A \wedge \neg B) \vee B \vee \neg A$$

Passo 3 — distributiva (sobre):

$$= (A \quad B \quad \neg A) \quad (\neg B \quad B \quad \neg A)$$

Ambas as cláusulas contêm um literal e seu complementar (A e $\neg A$ na primeira; B e $\neg B$ na segunda), portanto **ambas são tautologias** — o que confirma que a fórmula original é uma tautologia (é sempre verdadeira, como esperado, pois é a lei da contrapositiva).

FNC final:

$$(A \quad B \quad \neg A) \quad (\neg A \quad B \quad \neg B)$$

Questão 11

$$\neg P \quad \neg Q \quad R \quad S$$

Isolando R e S como conclusões (lado esquerdo do $\leftarrow$) e P, Q como condições:

$$R, S \leftarrow P, Q$$

Questão 12

$$Q \leftarrow R, S$$

$$R \leftarrow T$$

$$S \leftarrow$$

$$T \leftarrow$$

Prova por refutação (nega-se a meta e busca-se a cláusula vazia):

$\leftarrow Q$	(pergunta inicial)
$\leftarrow R, S$	(usa $Q \leftarrow R, S$)
$\leftarrow T, S$	(usa $R \leftarrow T$)
$\leftarrow S$	($T \leftarrow$ é fato, elimina T)
$\leftarrow$	($S \leftarrow$ é fato, elimina S)

Chegou-se à cláusula vazia $\leftarrow$, portanto **Q é derivado** do conjunto de cláusulas.

Questão 13

```
irmao_de(X, Y) :-
    pai(P, X), pai(P, Y), X \= Y.
irmao_de(X, Y) :-
    mae(M, X), mae(M, Y), X \= Y.
```

```
tio_ou_tia(X, Y) :-
    ( pai(P, Y) ; mae(P, Y) ),
    irmao_de(X, P).
```

Ideia: X é tio/tia de Y se existe um pai/mãe P de Y tal que X é irmão(ã) de P. `irmao_de/2` já garante $X \neq P$ (não é a mesma pessoa), então não é preciso checar de novo que X não é pai/mãe de Y diretamente — mas repare que isso só está correto se `irmao_de` não incluir P como irmão de si mesmo, o que já é garantido por $X \neq Y$ dentro dele.

Questão 14

```
primos(X, Y) :-
    ( pai(PX, X) ; mae(PX, X) ),
    ( pai(PY, Y) ; mae(PY, Y) ),
    irmao_de(PX, PY),
    X \= Y.
```

Ideia: busca um dos pais de X (PX) e um dos pais de Y (PY); se PX e PY são irmãos entre si, então X e Y são primos.

Questão 15

Trace: para a lista `[a,b,a,c,a]`, `member(X,...)` gera, em ordem, $X=a$, $X=b$, $X=a$, $X=c$, $X=a$ (repetindo conforme aparece na lista). Para **cada** valor gerado de X, `conta(X, [a,b,a,c,a], N)` é recalculado do zero contando quantas vezes aquele X aparece na lista **inteira**.

- $X=a \rightarrow$ conta 3 ocorrências de a $\rightarrow N=3$
- $X=b \rightarrow$ conta 1 ocorrência de b $\rightarrow N=1$
- $X=a$ (2ª vez do member) $\rightarrow$ de novo $N=3$
- $X=c \rightarrow N=1$
- $X=a$ (3ª vez) $\rightarrow$ de novo $N=3$

Resultado: `L = [3, 1, 3, 1, 3]`

O corte na segunda cláusula de `conta/3` evita que, ao casar a cabeça da lista com X, o Prolog também tente (por engano) a terceira cláusula em backtracking, o que contaria o mesmo elemento duas vezes.

Questão 16

```
triplas_pitagoricas(N, L) :-
    findall([A,B,C],
        ( between(1,N,C),
          between(1,C,B),
          between(1,B,A),
          A*A + B*B == C*C
```

```

    ),
    L).

?- triplas_pitagoricas(15, L).
L = [[3,4,5],[6,8,10],[5,12,13],[9,12,15]].

```

Questão 17

```

p(1). p(2). p(3).
q(X,Y) :- p(X), p(Y), X \= Y.

```

O produto cartesiano de $\{1,2,3\} \times \{1,2,3\}$ tem 9 combinações; a condição $X \neq Y$ remove as 3 em que $X=Y$, sobrando **6 soluções**:

```

X=1,Y=2
X=1,Y=3
X=2,Y=1
X=2,Y=3
X=3,Y=1
X=3,Y=2

```

(ordem: X varia mais lentamente, Y mais rapidamente, pois $p(Y)$ é tentado por completo antes de $p(X)$ avançar — backtracking é “da direita para a esquerda primeiro”).

Questão 18

```

quad(X, Y) :- Y is X*X.

?- maplist(quad, [1,2,3,4], L).
L = [1, 4, 9, 16].

```

Questão 19

```

oper(entrarSala1, [2,Y,Z], [1,Y,Z]).
oper(entrarSala2, [1,Y,Z], [2,Y,Z]).
oper(aspirar, [1,1,Z], [1,0,Z]).
oper(aspirar, [2,Y,1], [2,Y,0]).

```

(seguindo exatamente a notação `oper(,s,s')` do material — aqui como fatos, já que as condições estão embutidas nos próprios padrões dos argumentos.)

Questão 20

Após as três chamadas de **incrementa**, **contador(X)** terá **X = 3**.

Por quê não há duplicação: cada chamada de **incrementa** primeiro **remove** (**retract**) o fato atual **contador(N)** da base — deixando zero cláusulas de **contador/1** momentaneamente — e só depois **insere** (**assert**) o novo fato **contador(N1)**. Como o **retract** sempre acontece antes do **assert** dentro da mesma chamada, nunca existem dois fatos **contador/_** ao mesmo tempo: a base sempre tem exatamente um fato de **contador/1**, atualizado a cada chamada ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3$).